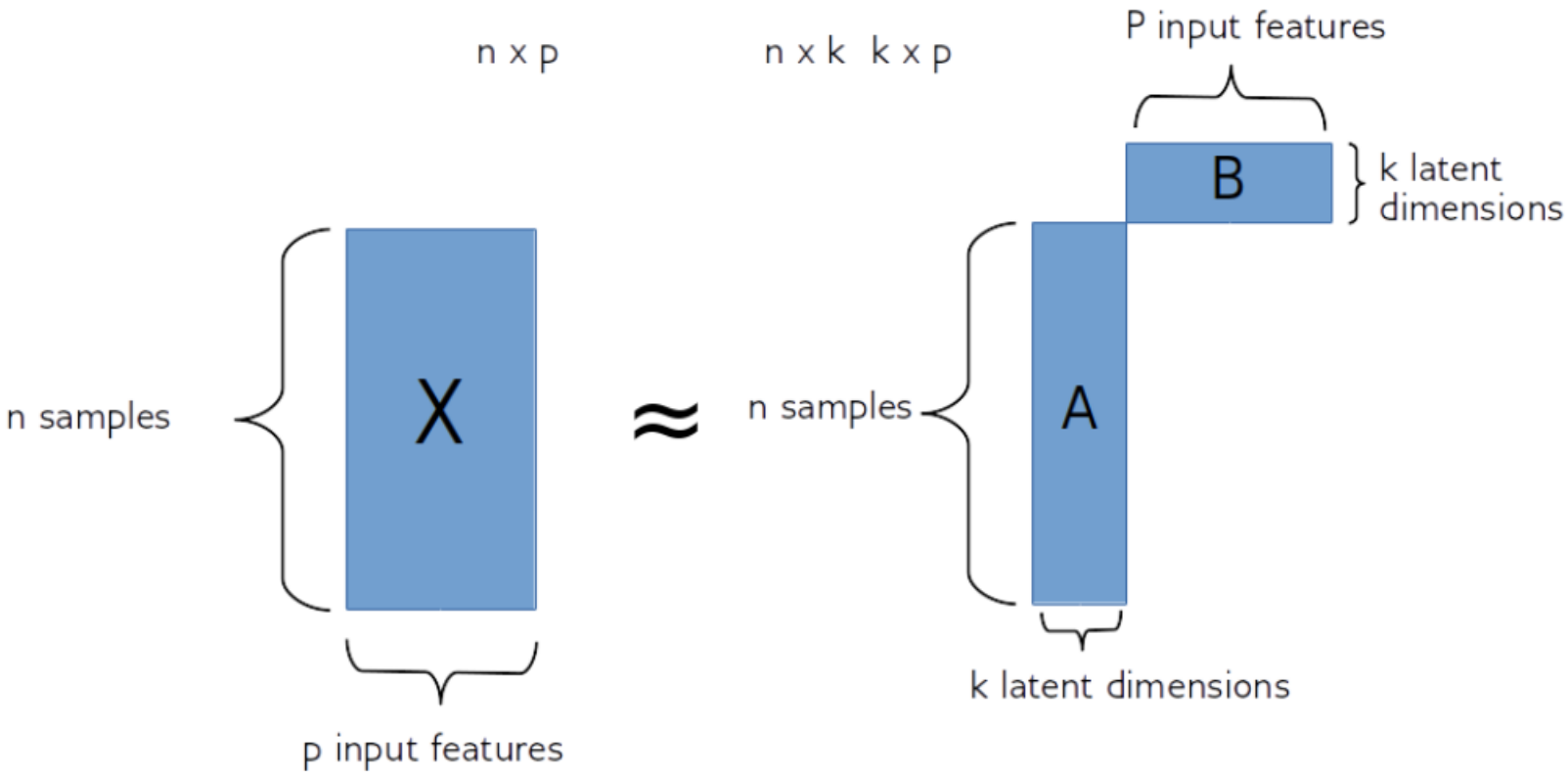
Non-Negative Matrix Factorization

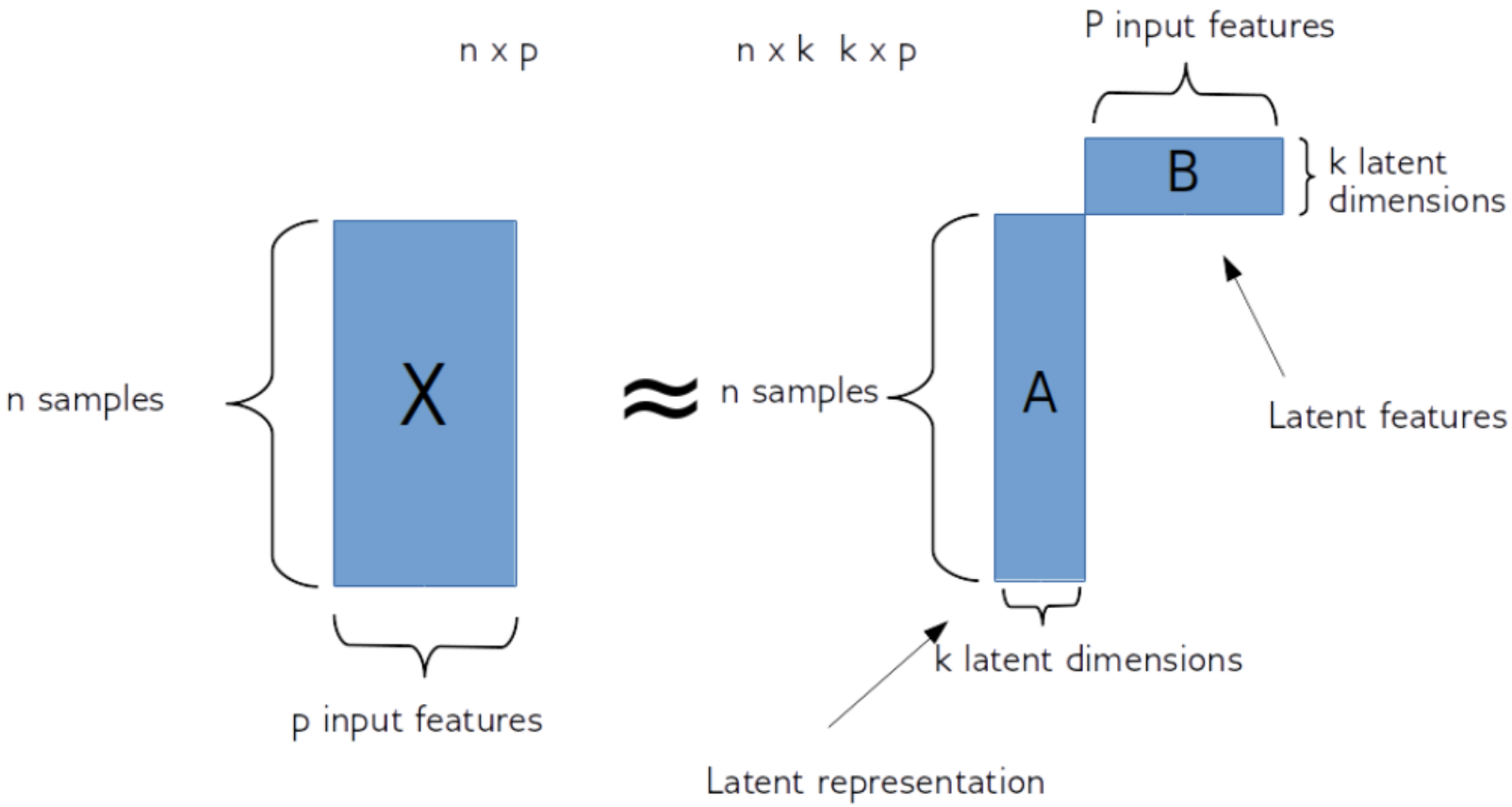
Detección de anomalías

Non-Negative Matrix Factorization

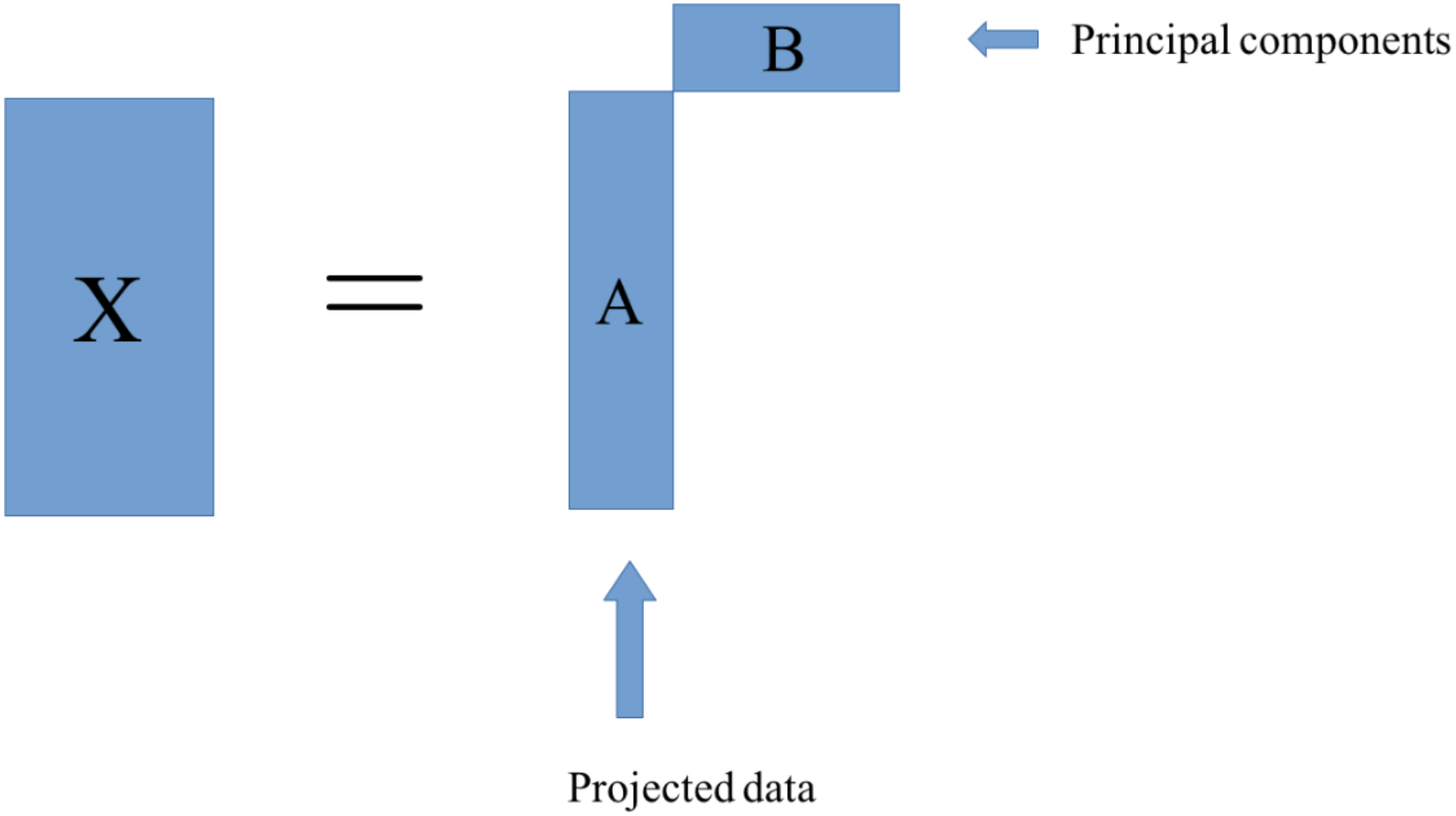
$$\underset{n \times p}{X} \approx \underset{n \times k}{A}\,\underset{k \times p}{B}$$

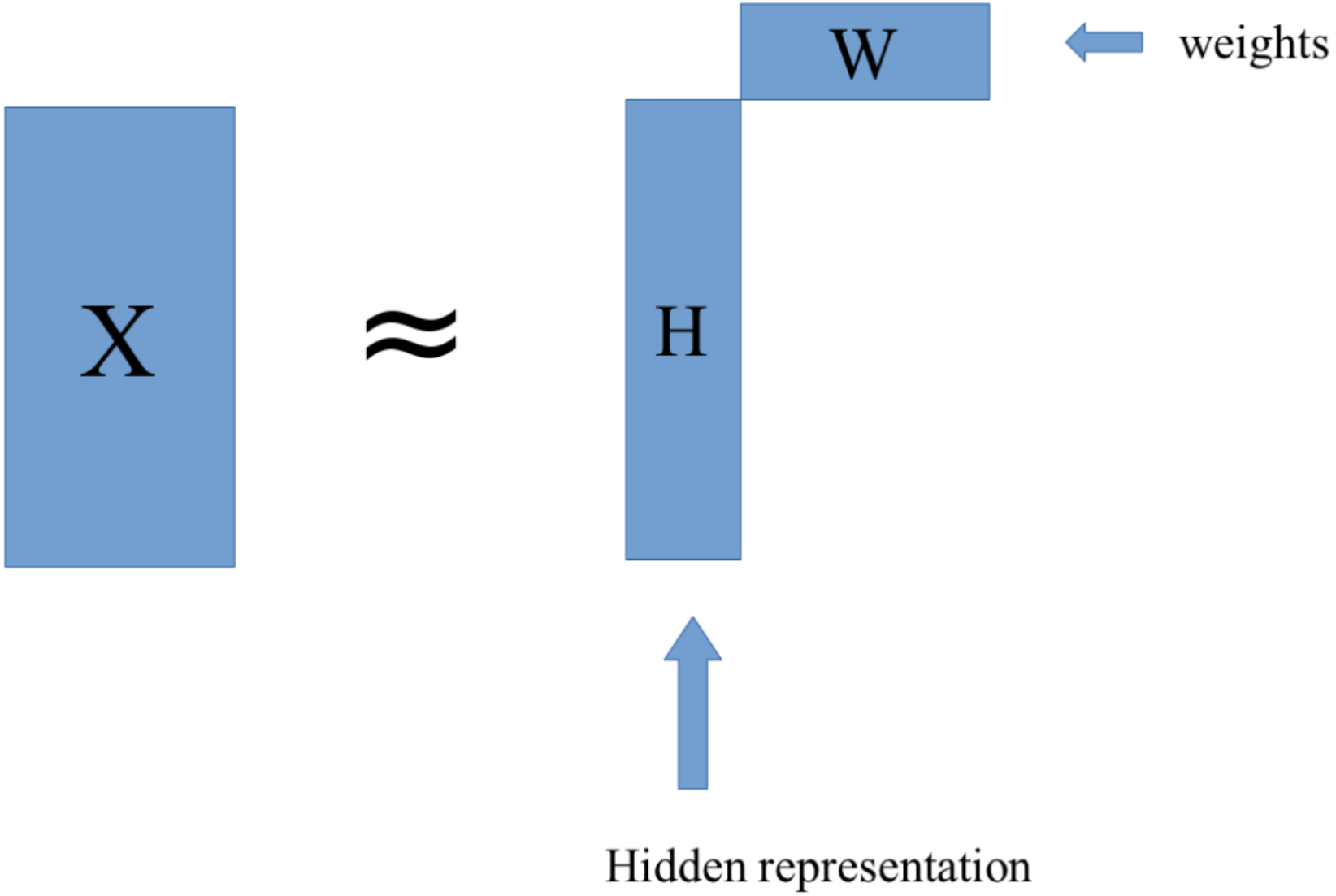


$$\underset{n \times p}{X} \approx \underset{n \times k}{A}\,\underset{k \times p}{B}$$



sklearn speak: `A = mf.transform(X)`

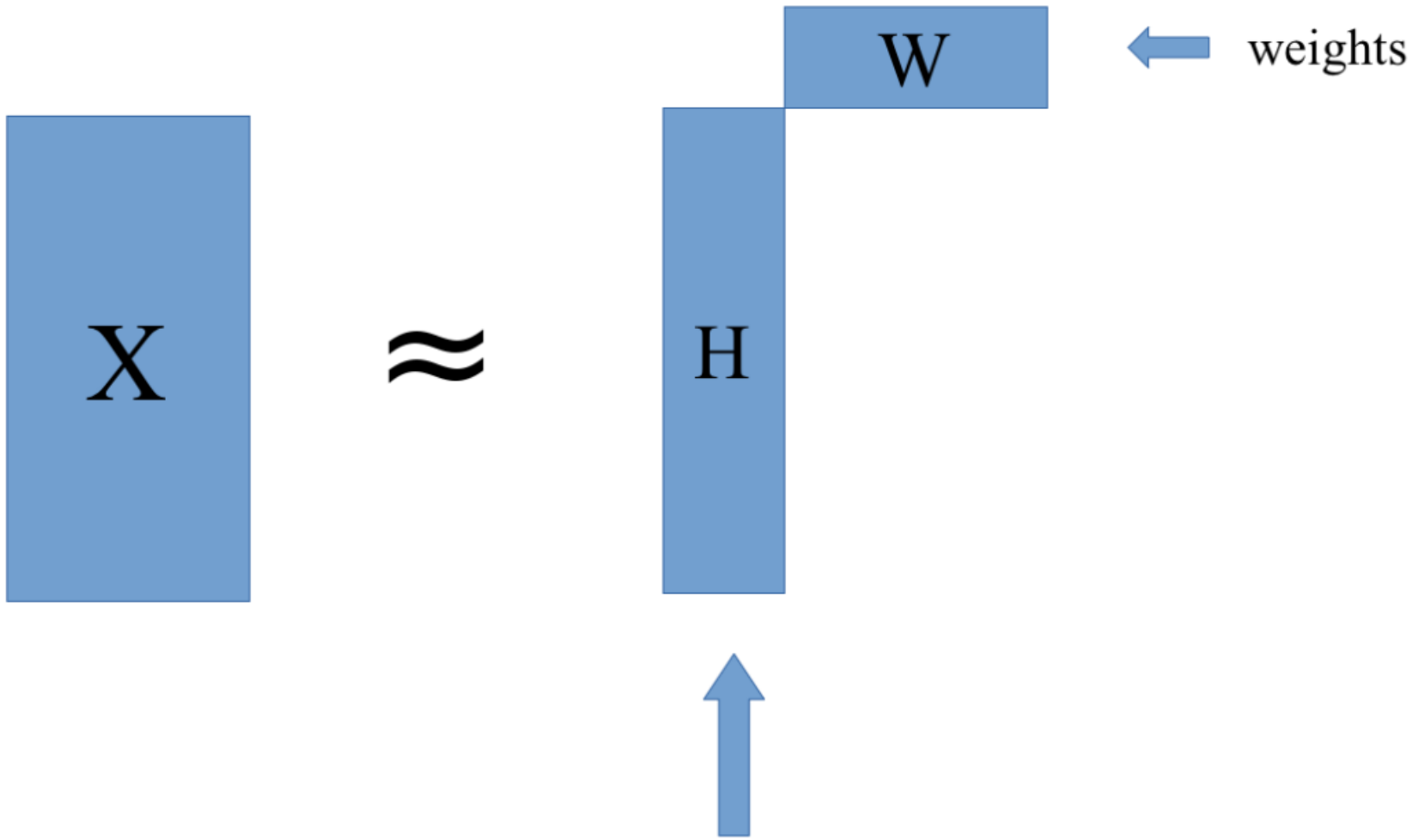

B
Principal components
X
=
A
Projected data


W
weights
X
≈
H
Hidden representation

Otras Técnicas de factorización de matrices

- PCA: componentes principales ortogonales, minimizar la pérdida al cuadrado
- PCA disperso: componentes ortogonales y dispersos
- ICA: componentes independientes
- Factorización de la matriz no negativa (NMF): la representación latente y las características latentes son no negativas.

NMF



Pérdida y algoritmo NMF

Frobenius loss / squared loss ($Y = HW$):

$$l_{\text{frob}}(X, Y) = \sum_{i,j} (X_{ij} - Y_{ij})^2$$

Kulback-Leibler (KL) divergence:

$$l_{\text{KL}}(X, Y) = \sum_{i,j} X_{ij} \log\left(\frac{X_{ij}}{Y_{ij}}\right) - X_{ij} + Y_{ij}$$

optimización :

Convexo en W o H, no en ambos.
Inicialización aleatoria (¿PCA?)
Actualizar iterativamente W y H (descenso de coordenadas en bloque)

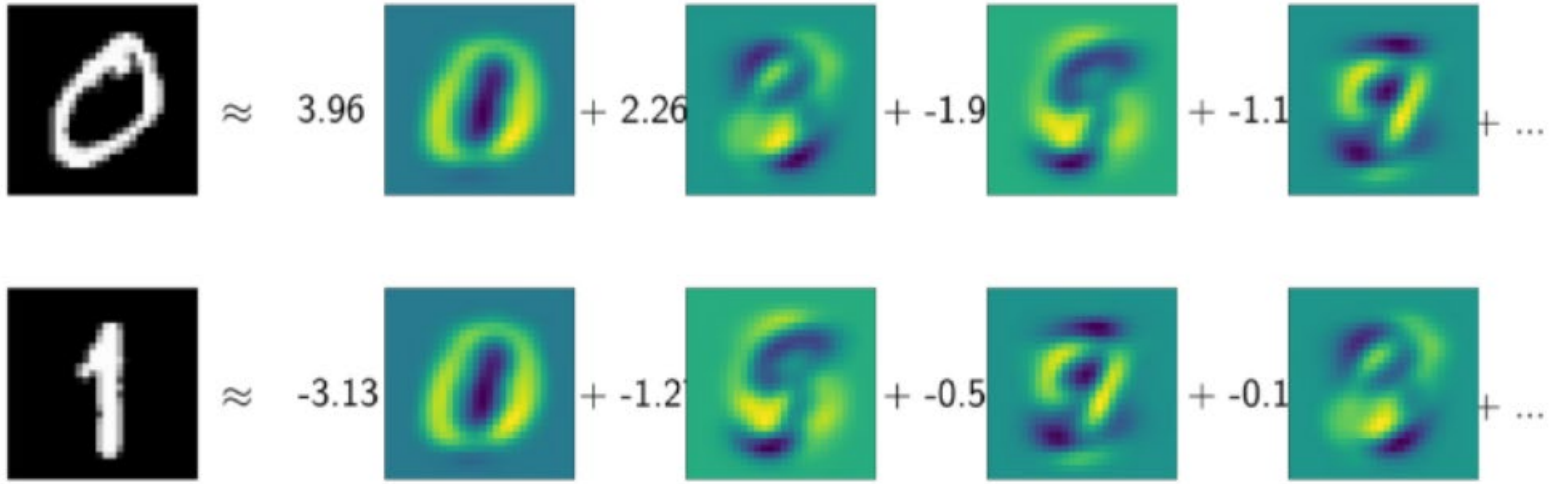
Transformación de los datos:

Dados los nuevos datos X_test, W fijo, el cálculo de H requiere una optimización.

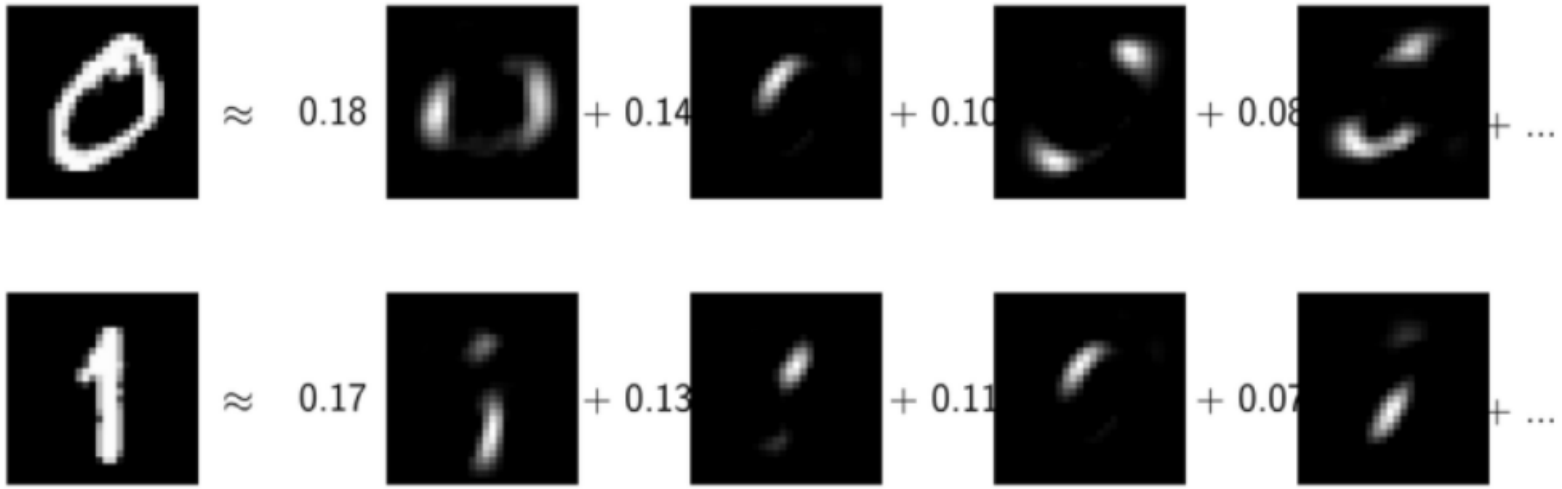
¿Por qué NMF?

- Signos significativos
- Pesos positivos
- No hay "cancelación" como en el PCA
- Puede aprender una representación excesivamente completa

PCA (ordenado por proyección, no por valor propio)

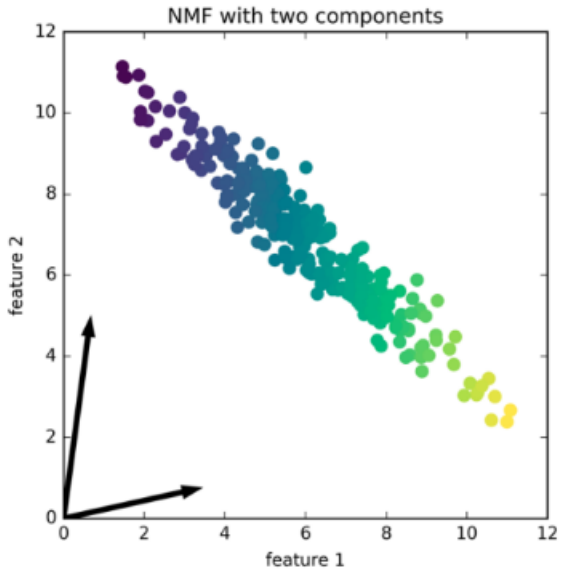


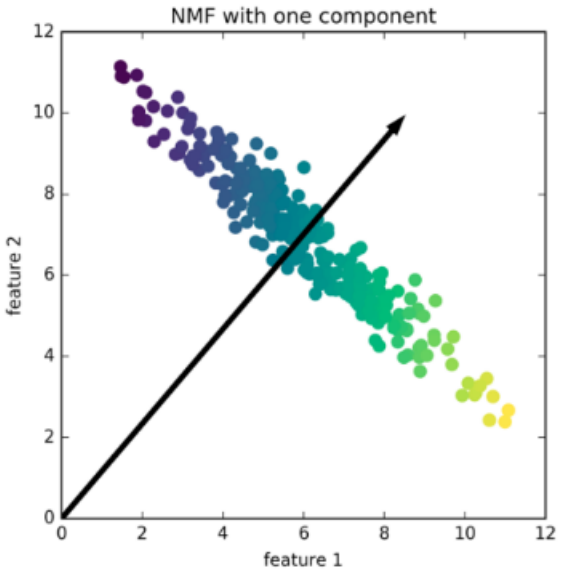
NMF (ordenado por representación oculta)



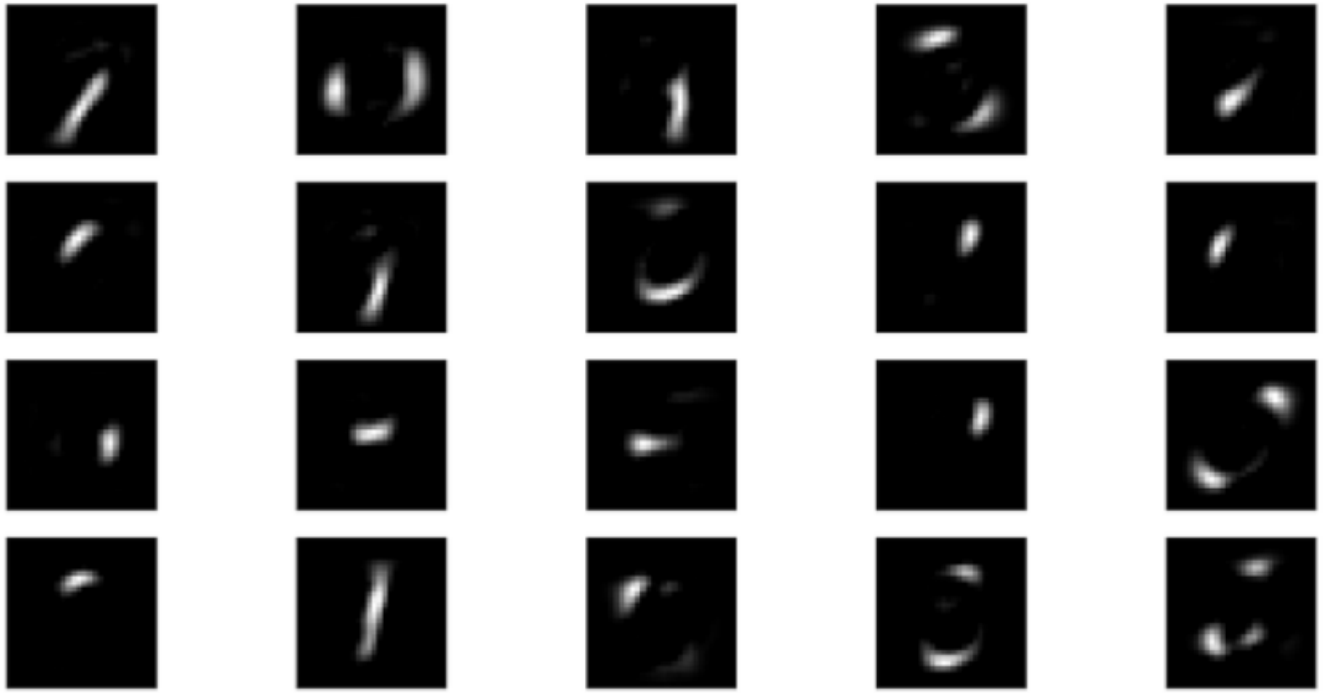
Desventajas del NMF

- Sólo puede aplicarse a datos no negativos
- La interpretabilidad es un acierto o un error
- Optimización no convexa, requiere inicialización
- No es ortogonal





NMF con 20 componentes en MNIST



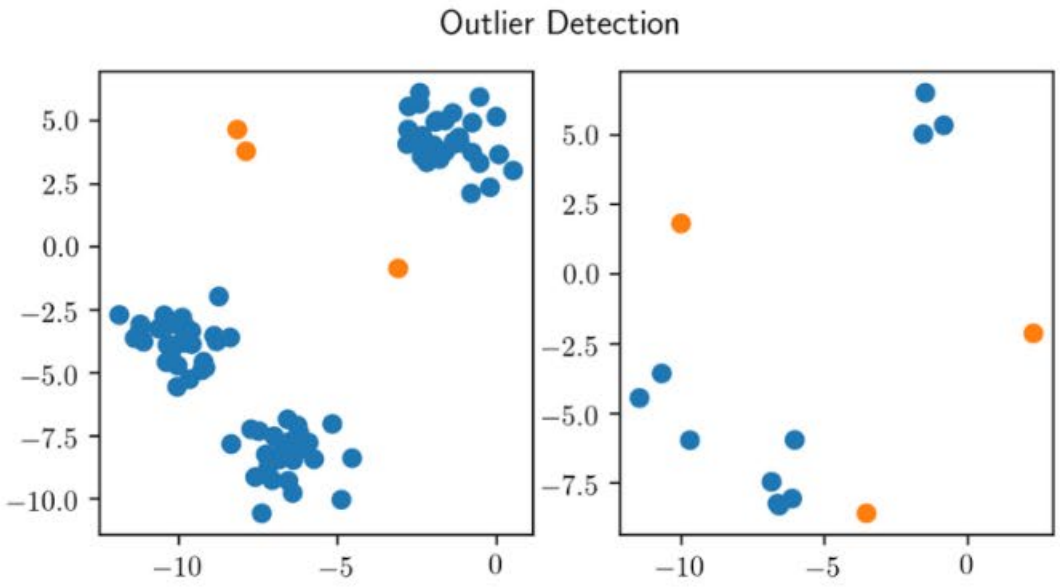
NMF con 5 componentes en MNIST

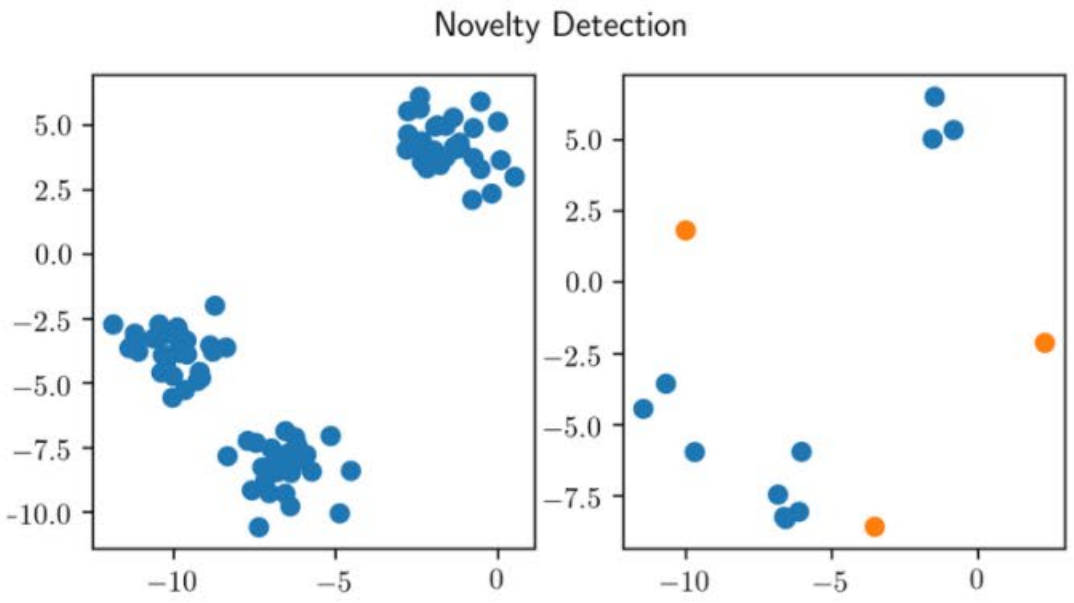
Aplicaciones del NMF

- Análisis de textos (próxima semana)
- Procesamiento de señales
- Voz y audio (ver librosa — librosa 0.9.1 documentation)
- Separación de fuentes
- Análisis de expresión genética

Detección de anomalías

Motivación


Outlier Detection
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10
−5
0


Novelty Detection
5.0
2.5
0.0
−2.5
−5.0
−7.5
-10.0
−10
−5
0

Aplicaciones

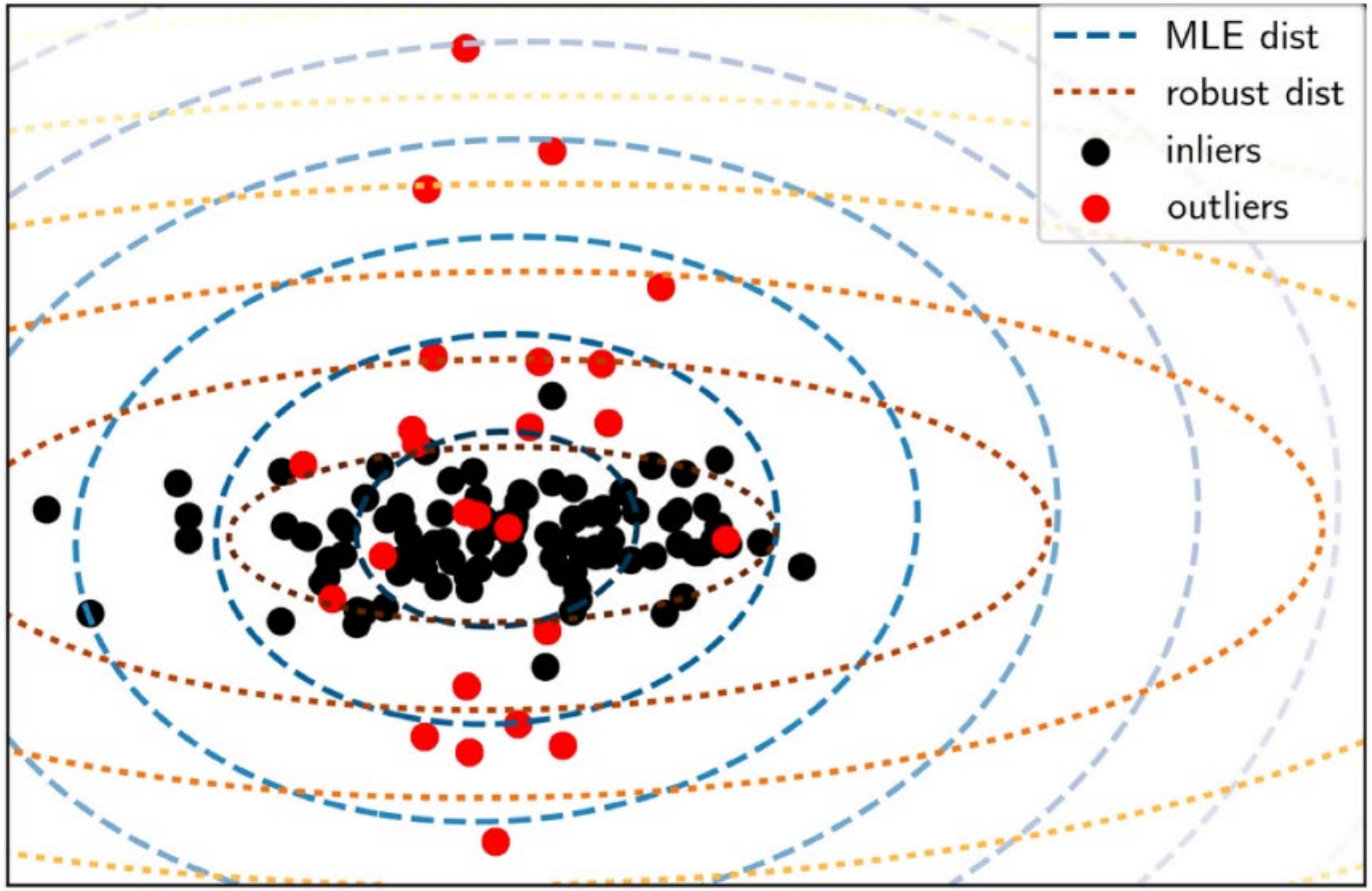
- Detección de fraudes (tarjetas de crédito, fraude por clic, ...)
- Detección de fallos en la red
- Detección de intrusiones en redes
- Detección de defectos (ingeniería, etc.)
- Noticias... ¿Inteligencia?

Idea básica

- Modelo de distribución de datos $p(X)$
- Valor atípico: $p(X) < \varepsilon$
- Para la detección de valores atípicos: ser robusto en la modelización de $p(X)$

Envoltura elíptica

$$p(X) = \mathcal{N}(\mu, \Sigma)$$



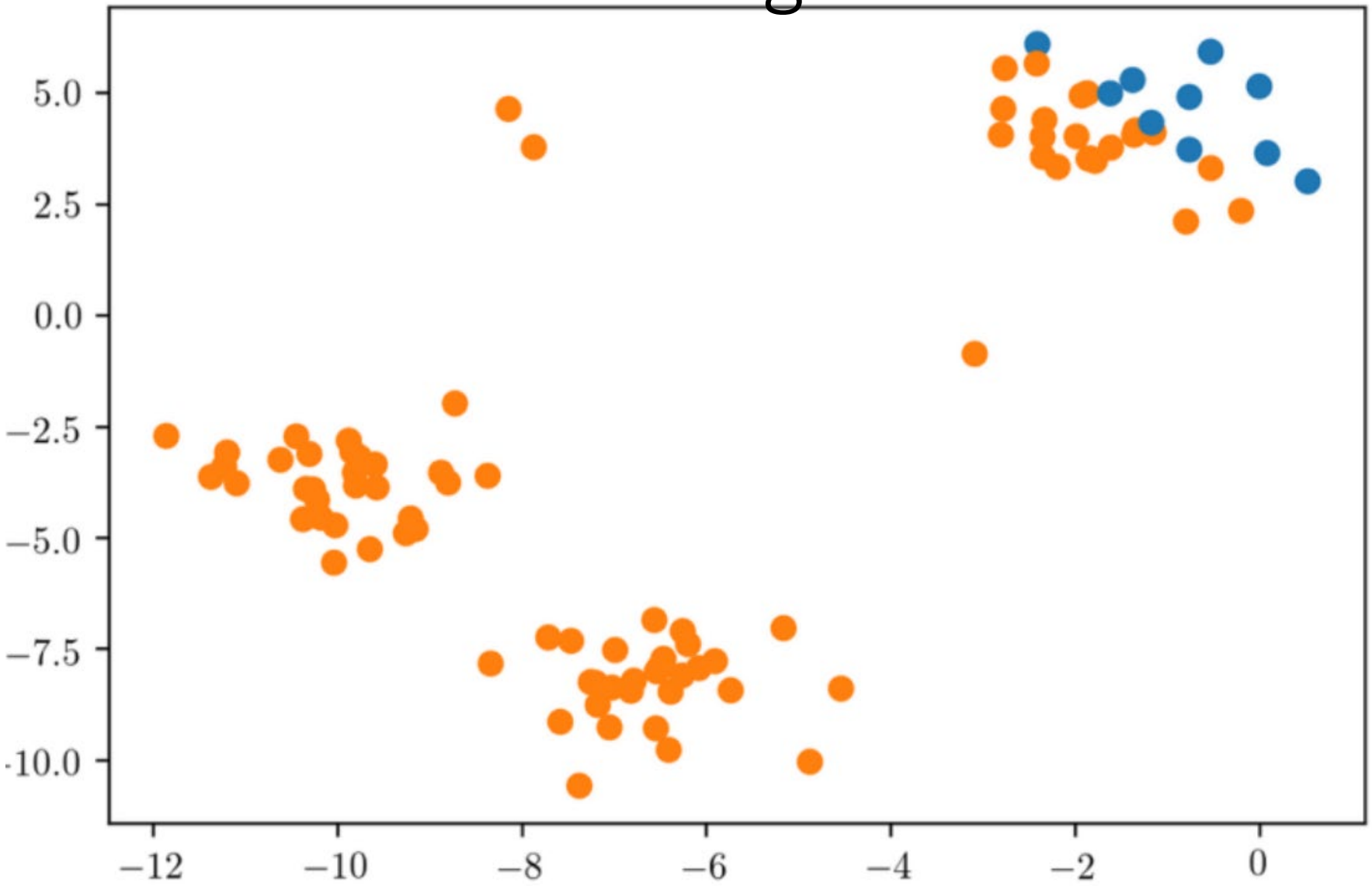
Envoltura elíptica

El preprocesamiento con PCA puede ayudar a veces.

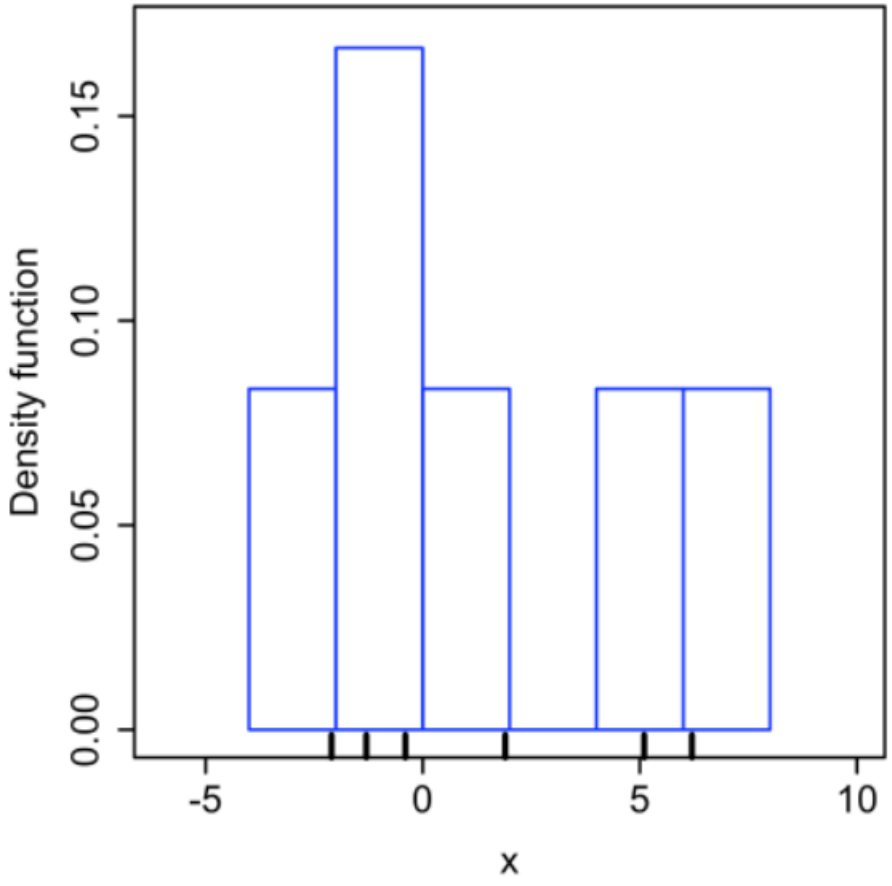
```
from sklearn.covariance import EllipticEnvelope
ee = EllipticEnvelope(contamination=.1).fit(X)
pred = ee.predict(X)
print(pred)
print(np.mean(pred == -1))
```

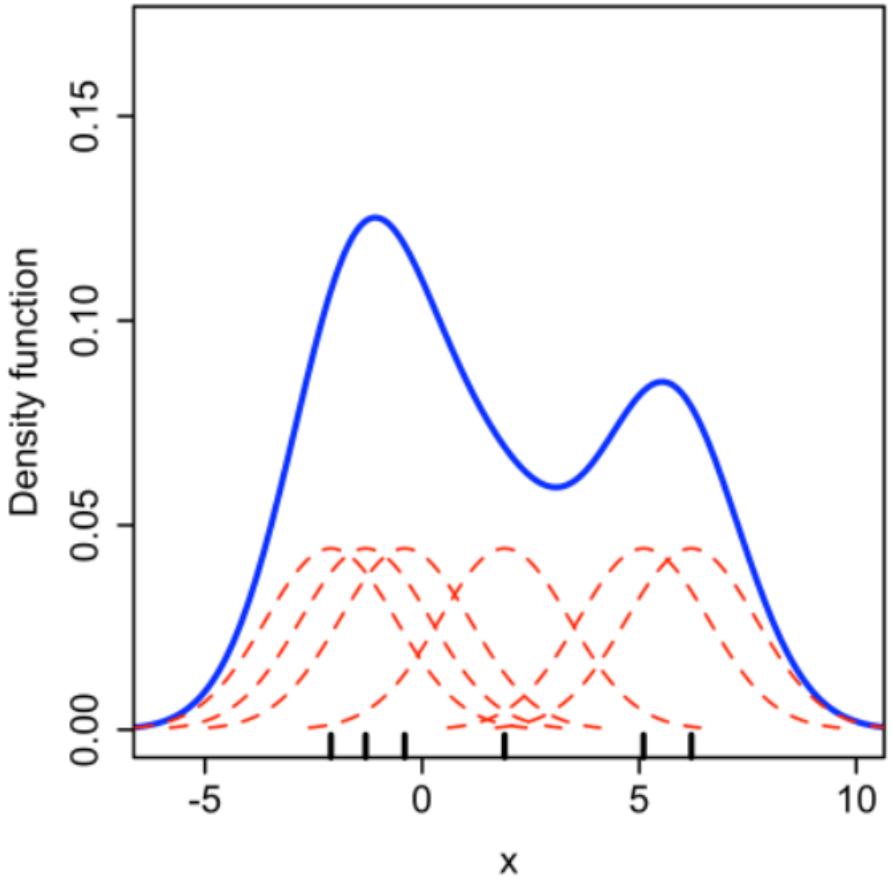
```
[ 1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
 -1  1  1  1 -1 -1  1 -1  1  1 -1  1 -1 -1 -1  1  1 -1 -1 -1 -1  1 -1  1  1]
0.104
```

Caso de fallo: Datos no gaussianos

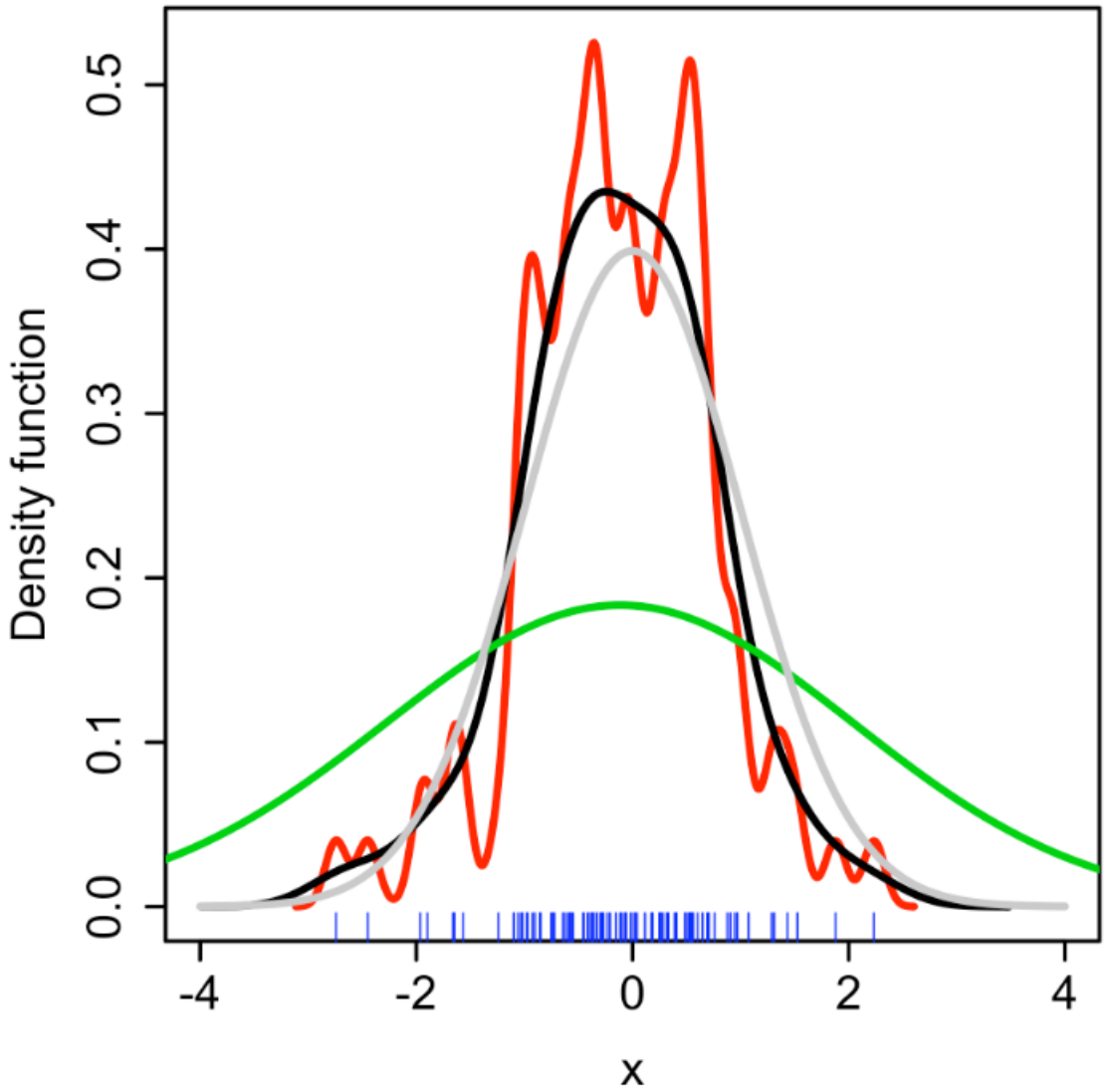


Kernel Density

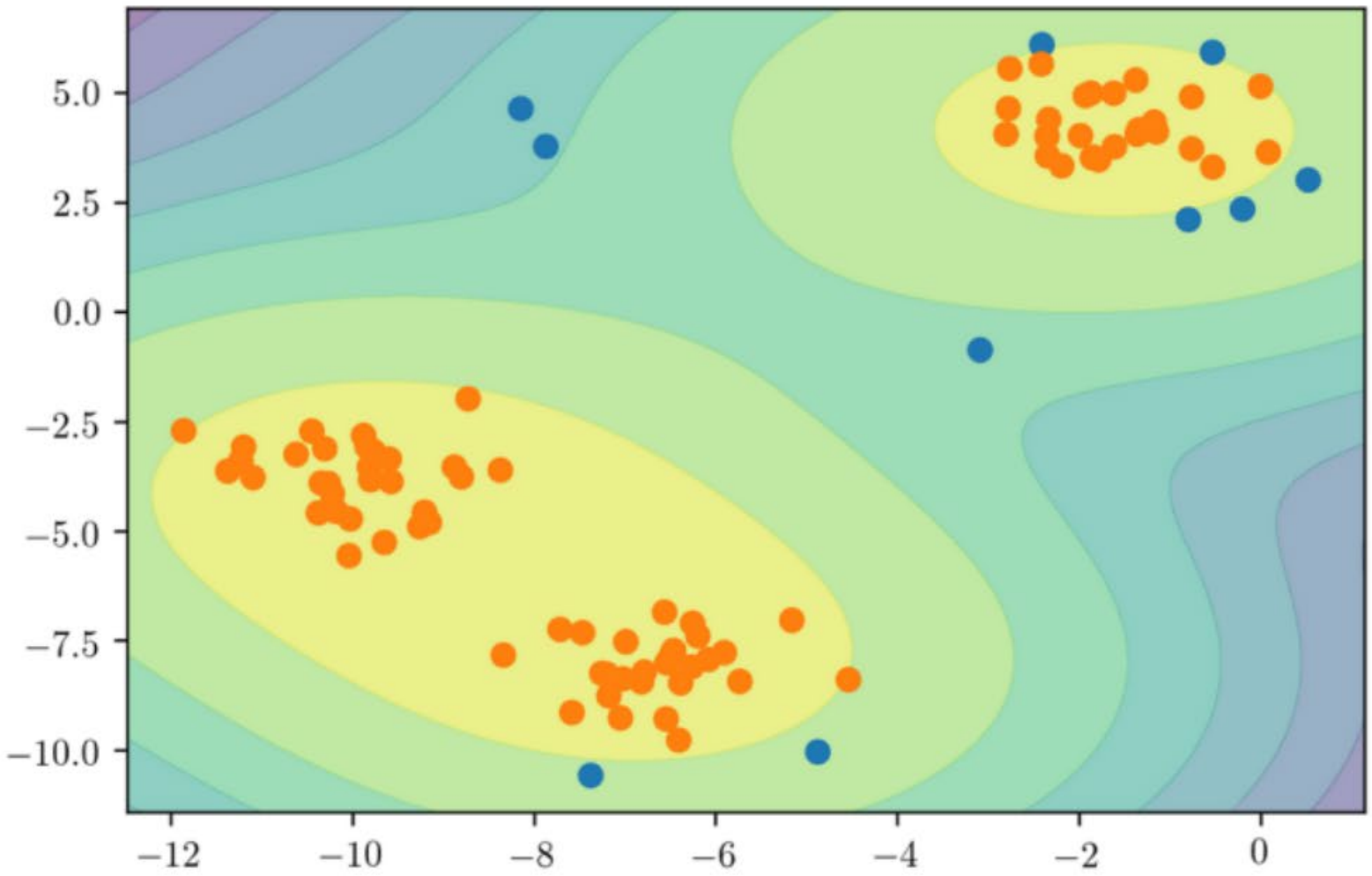




Kernel Bandwidth



```
kde = KernelDensity(bandwidth=3)
kde.fit(X_train_noise)
pred = kde.score_samples(X_train_noise)
pred = (pred > np.percentile(pred, 10)).astype(int)
```



SVM de una clase

Original *SVM* Formulation:

$$\min_{w,b,\zeta} \frac{1}{2} w^T w + C \sum_{i=1}^{n} \zeta_i$$

$$\text{subject to } y_i(w^T \phi(x_i) + b) \geq 1 - \zeta_i,$$
$$\zeta_i \geq 0, i = 1, \ldots, n$$

$\nu - SVM$ Formulation:

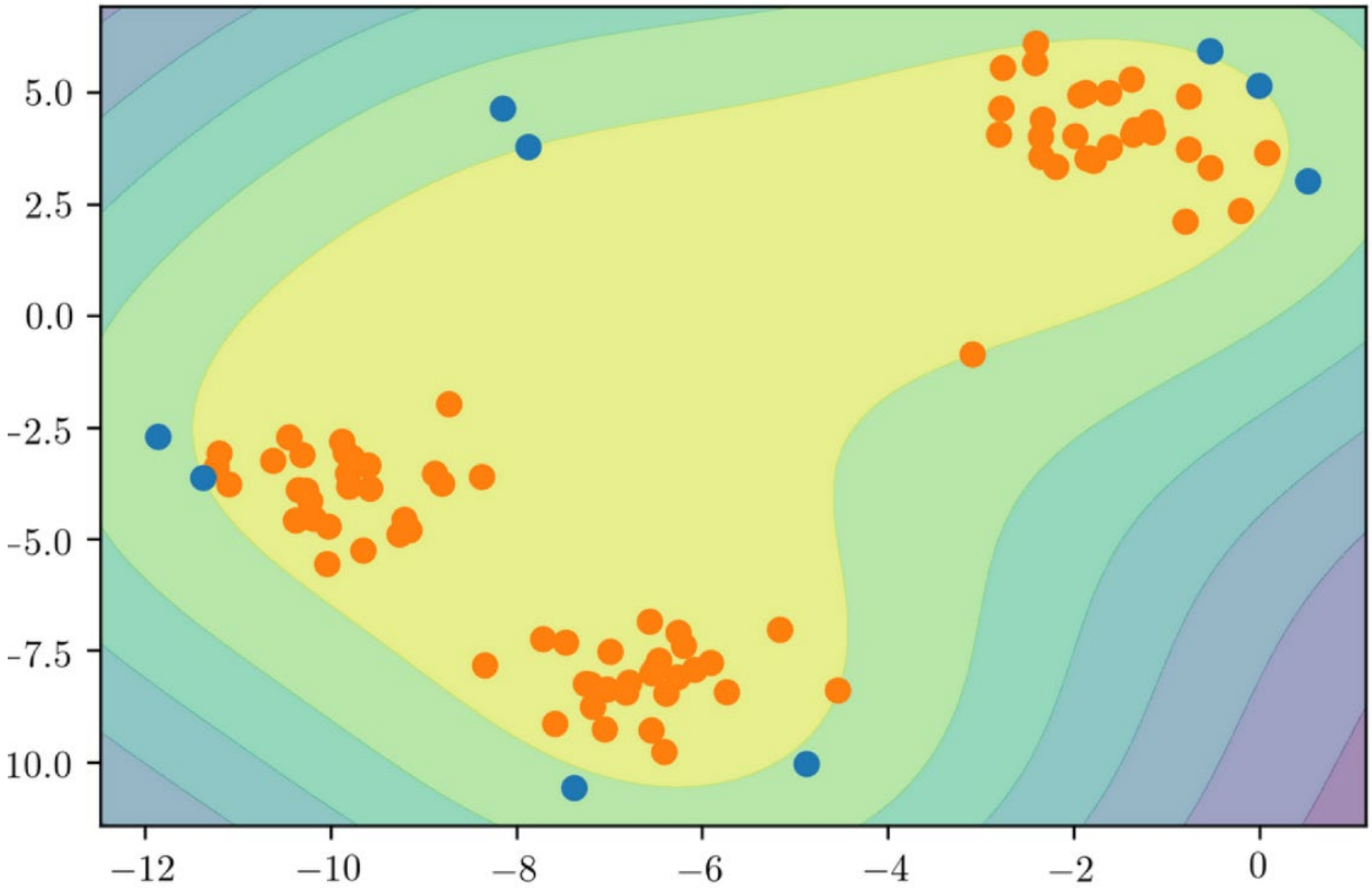
$$\min_{w \in F, \boldsymbol{\xi} \in \mathbb{R}^\ell, \rho \in \mathbb{R}} \quad \frac{1}{2}||w||^2 + \frac{1}{\nu \ell} \sum_i \xi_i - \rho$$

$$\text{subject to} \quad (w \cdot \Phi(\mathbf{x}_i)) \geq \rho - \xi_i, \quad \xi_i \geq 0$$

SVM de una clase

- También utiliza el kernel gaussiano para cubrir los datos
- Sólo selecciona los vectores de soporte (no todos los puntos)
- Especifica la proporción de valores atípicos (contaminación) mediante **nu**

```
from sklearn.svm import OneClassSVM
scaler = StandardScaler()
X_train_noise_scaled = scaler.fit_transform(X_train_noise)
oneclass = OneClassSVM(nu=.1).fit(X_train_noise_scaled)
pred = oneclass.predict(X_train_noise_scaled)
```

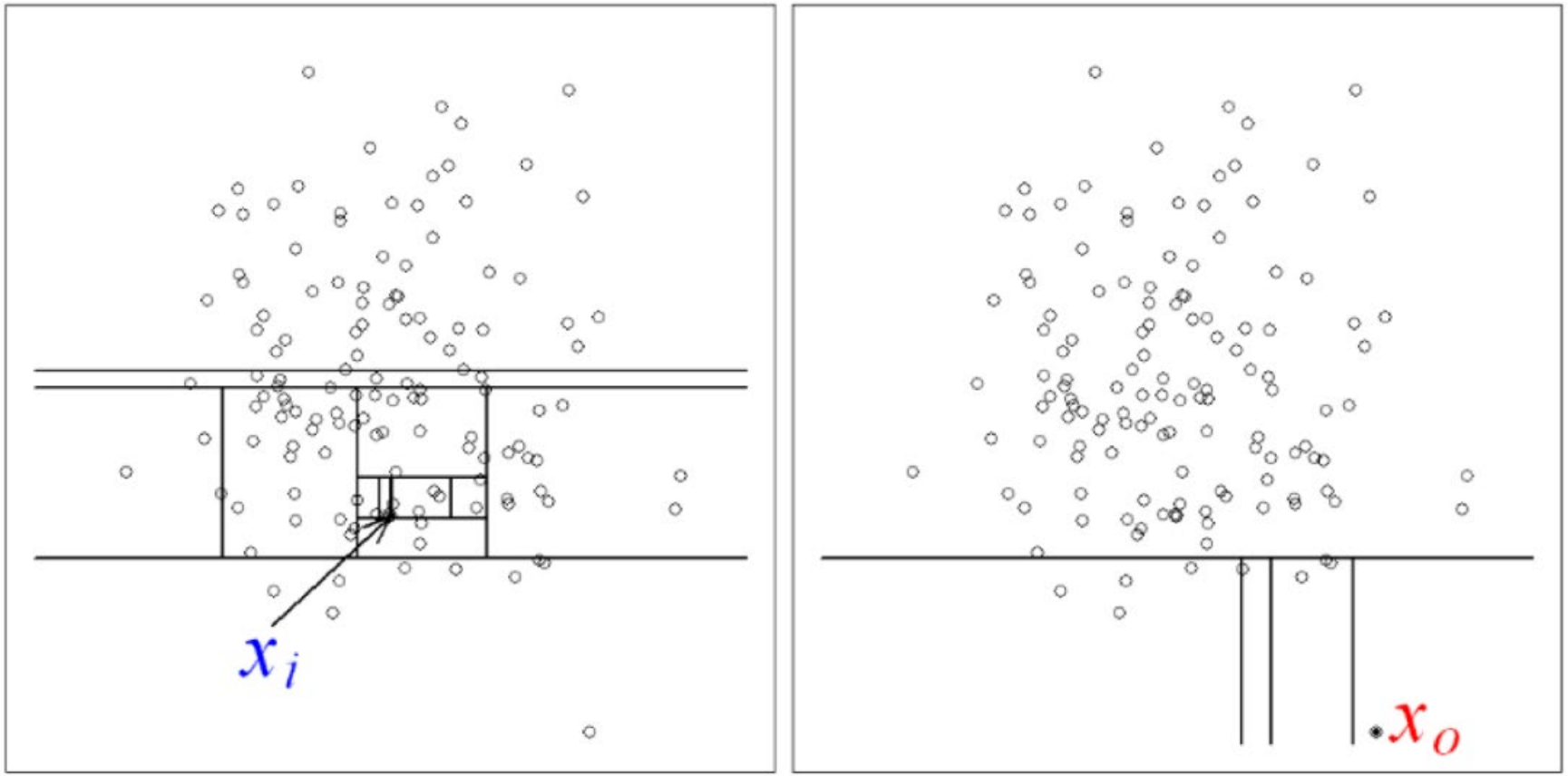

5.0
2.5
0.0
-2.5
-5.0
-7.5
10.0
−12
−10
−8
−6
−4
−2
0

(74) Tutorial | Anomaly Detection | Local Outlier Factor | LOF Algorithm - YouTube

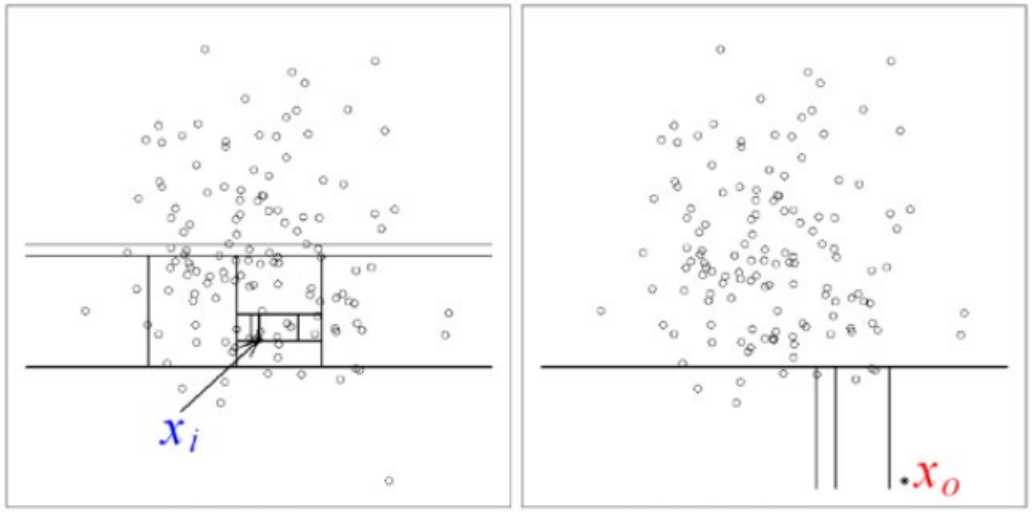
Isolation Forests

Idea

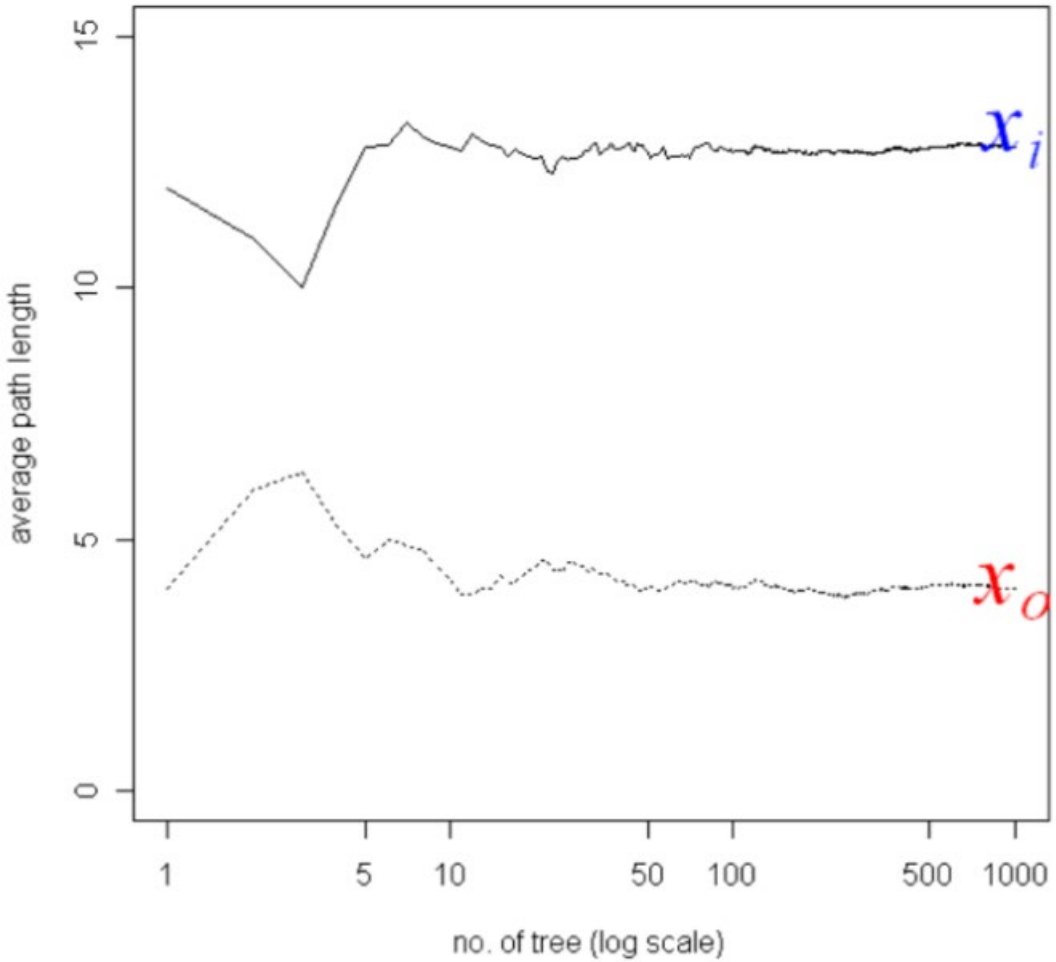
Los valores atípicos son fáciles de aislar del resto



(a) Isolating x_i (b) Isolating x_o



(a) Isolating x_i (b) Isolating x_o



Normalización de la longitud de la ruta

Longitud media de la ruta de búsqueda infructuosa en el árbol de búsqueda binario:

$$c(n) = 2H(n-1) - \left(\frac{2(n-1)}{n}\right) \text{ (H = Harmonic number)}$$

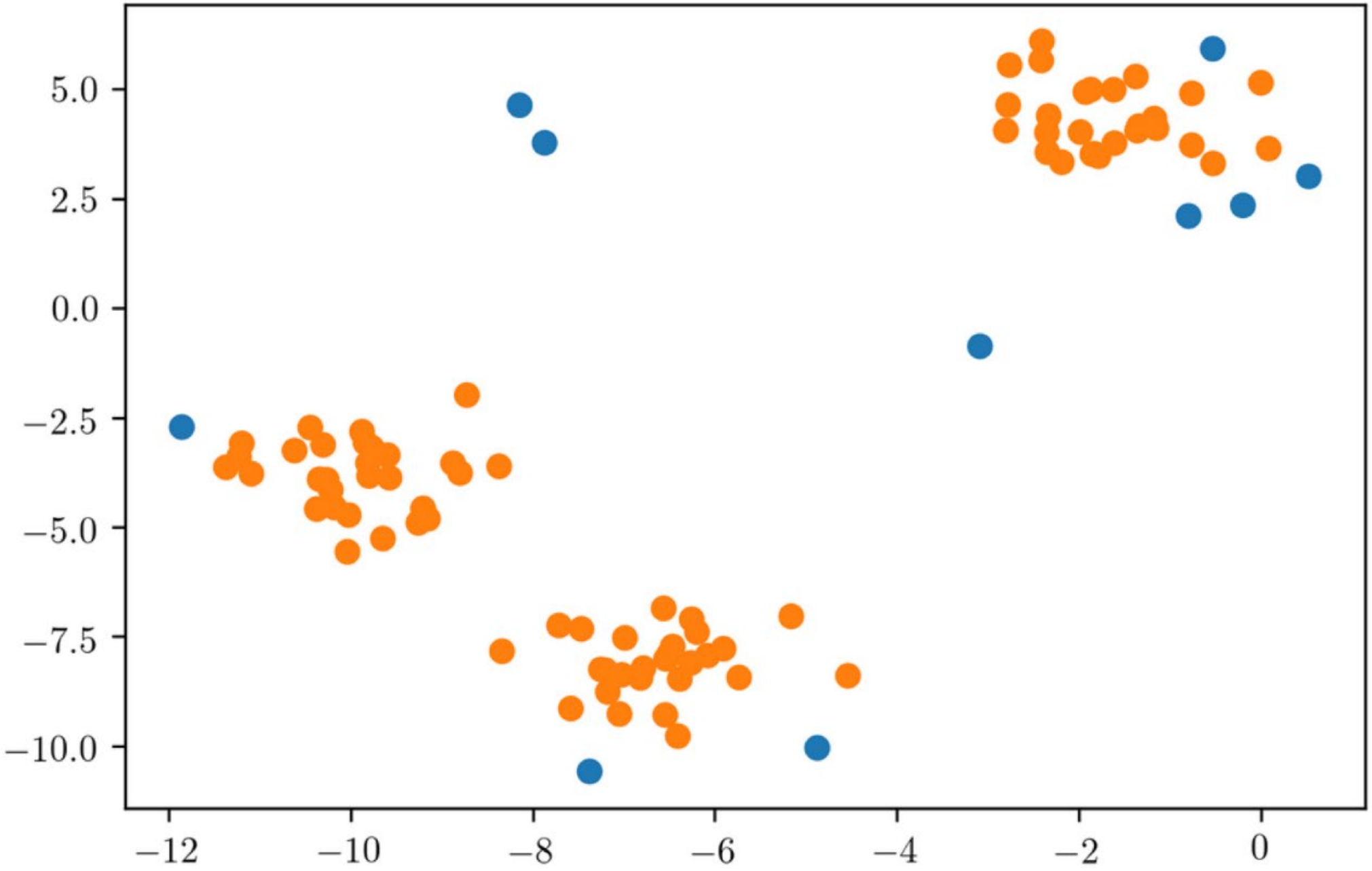
$$s(x,n) = 2^{-\frac{E(h(x))}{c(n)}} \text{ (h = depth in tree)}$$

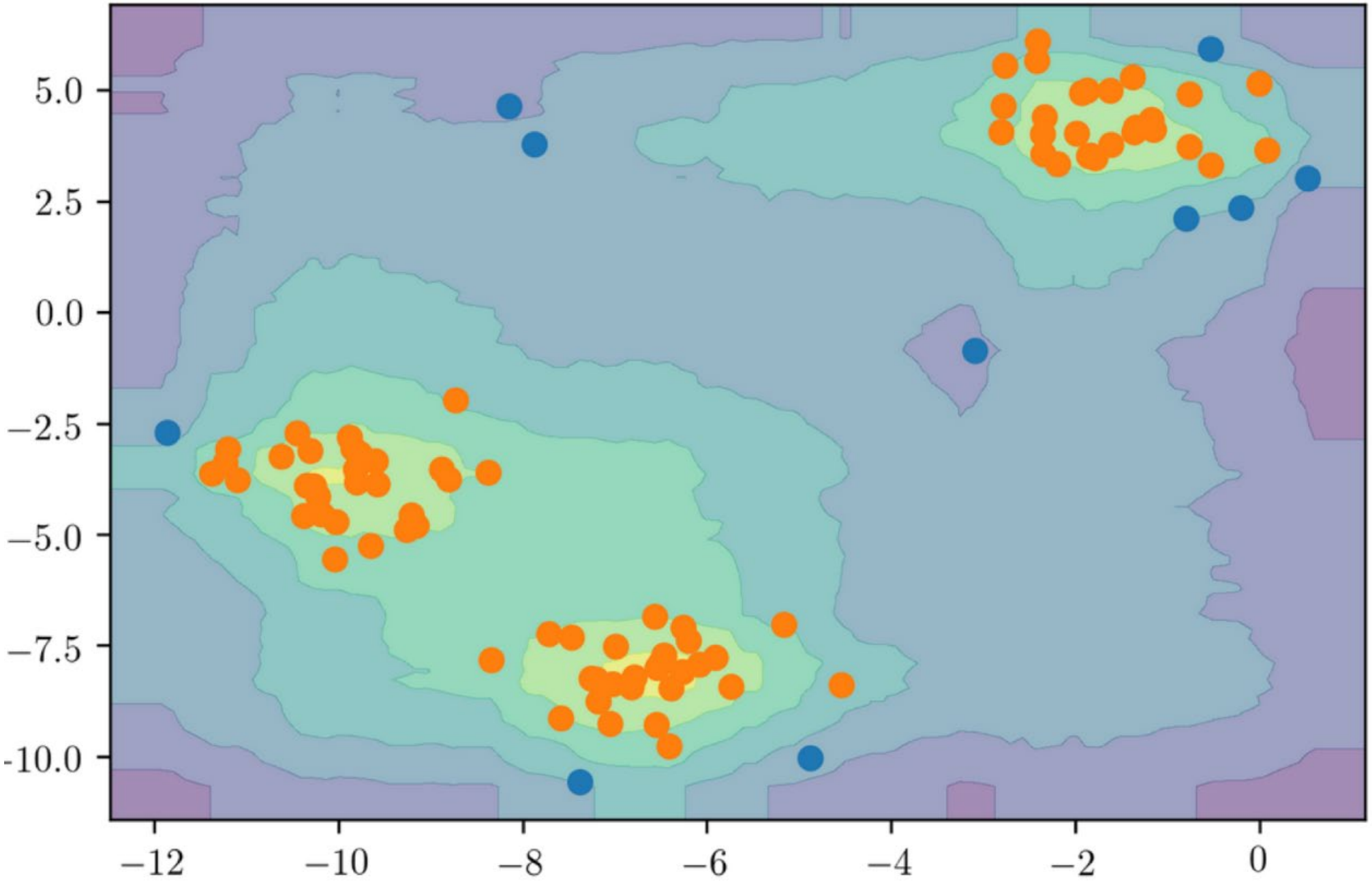
s < 0,5 : inlier definitivo

s cercano a 1: valor atípico

Construcción del bosque

- Conjunto de datos de submuestra para cada árbol
- El tamaño de muestra por defecto de 256 funciona sorprendentemente bien
- Detener el crecimiento del árbol en la profundidad log_2(tamaño de la muestra) - es decir, 8
- No se hace bootstrapping
- Más árboles es mejor - por defecto 100
- Es necesario especificar la tasa de contaminación

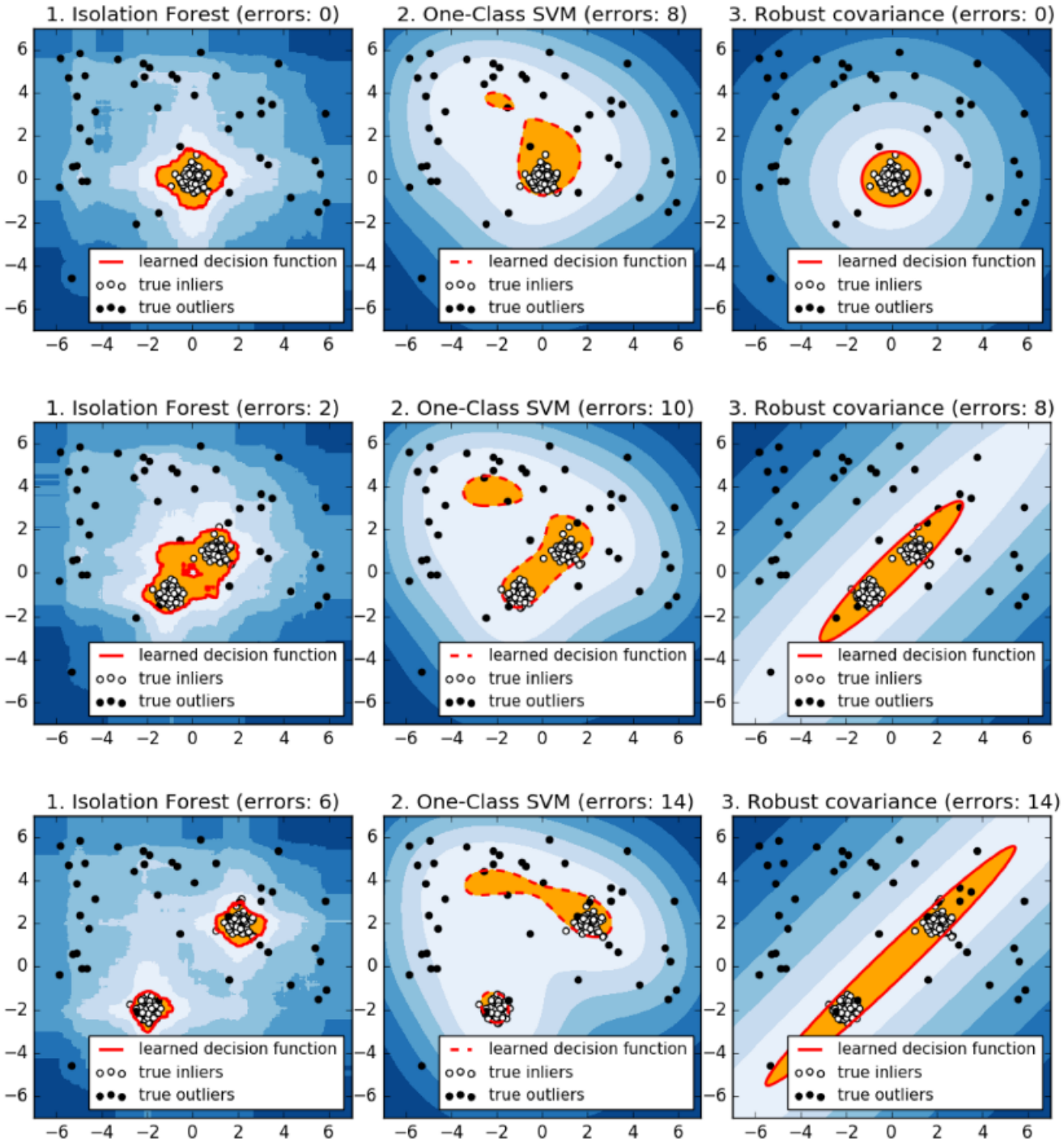

5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−12
−10
−8
−6
−4
−2
0


5.0
2.5
0.0
−2.5
−5.0
−7.5
-10.0
−12
−10
−8
−6
−4
−2
0

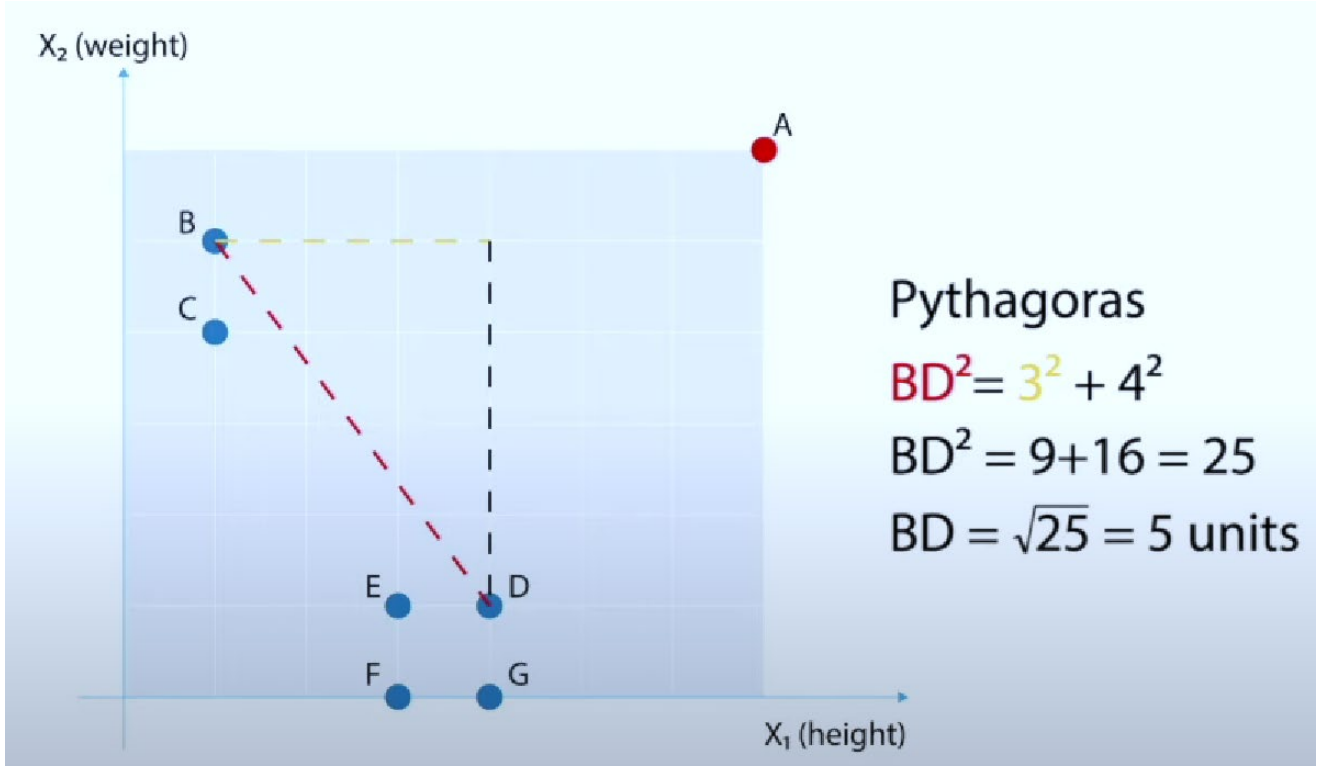
Otros modelos basados en la densidad

- PCA
- GMMs
- PCA robusto (no en sklearn :-()
- Cualquier otro modelo probabilístico - "robusto" es mejor.

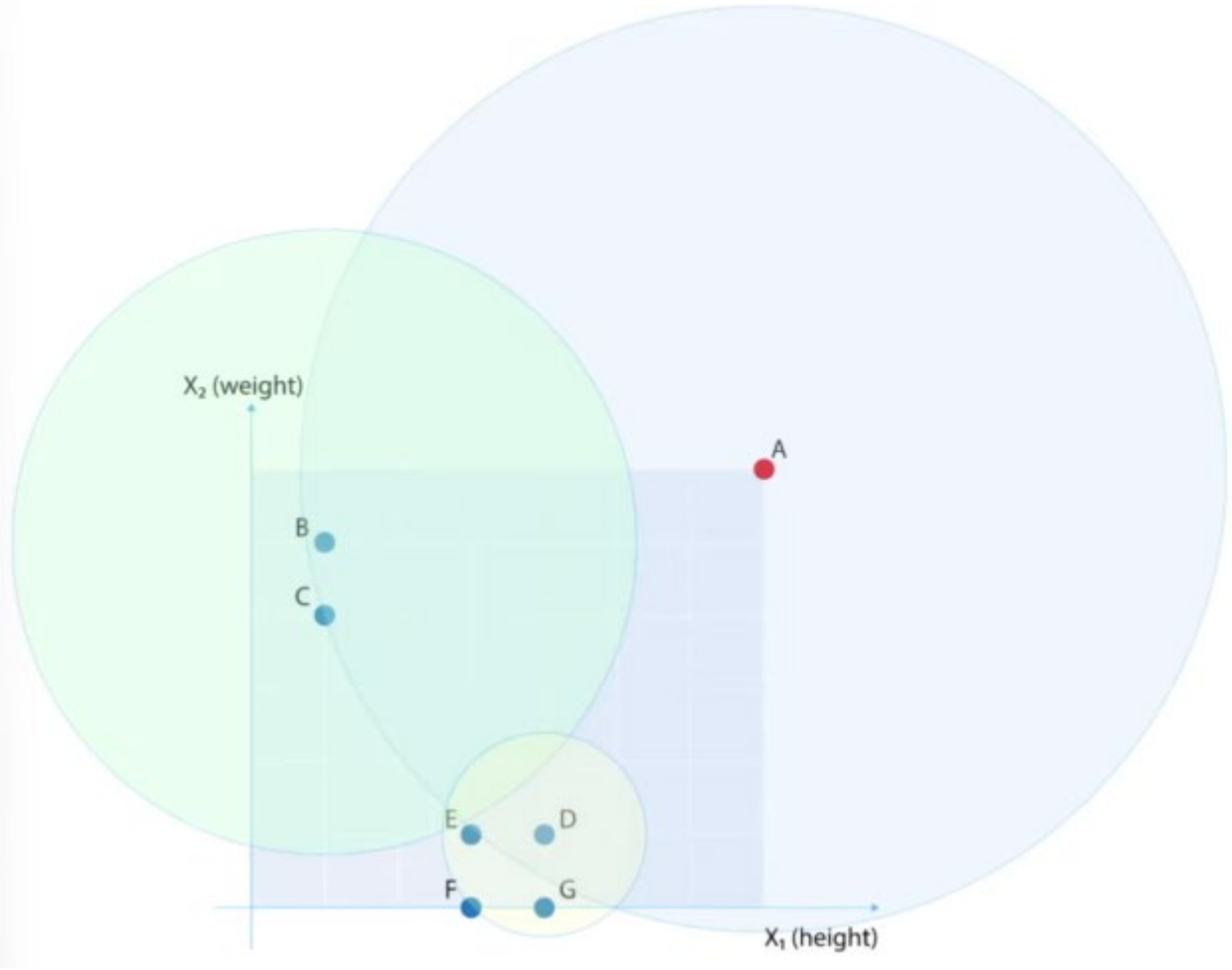
[4. Anomaly Detection - Hands-On Unsupervised Learning Using Python [Book] (oreilly.com)]


1. Isolation Forest (errors: 0)
2. One-Class SVM (errors: 8)
3. Robust covariance (errors: 0)
1. Isolation Forest (errors: 2)
2. One-Class SVM (errors: 10)
3. Robust covariance (errors: 8)
1. Isolation Forest (errors: 6)
2. One-Class SVM (errors: 14)
3. Robust covariance (errors: 14)
learned decision function
true inliers
true outliers

Local Outlier Factor (LOF)



Distances		
AB	6.08	2nd
AC	6.32	3rd
AD	5.8	1st
AE	6.4	
AF	7.2	
BA	6.08	
BC	1	1st
BD	5	3rd
BE	4.4	2nd
BF	5.3	
CA	6.32	
CB	1	1st
CD	4.2	3rd
CE	3.6	2nd
CF	4.4	
DA	5.8	
DB	5	
DC	4.2	
DE	1	1st
DF	1.4	3rd
DG	1	2nd



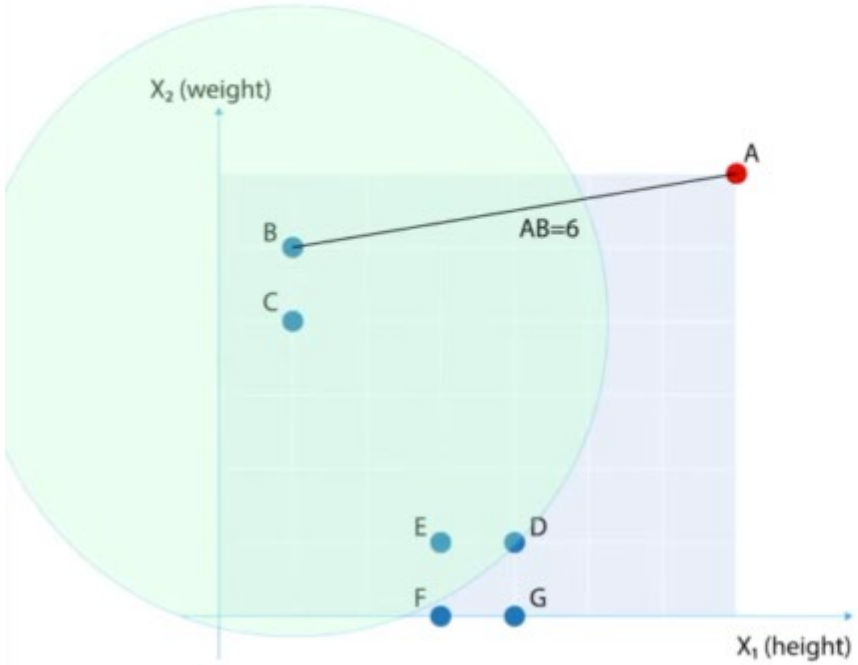
- Definir el hiper parámetro K
- Calcular el promedio de la alcanzabilidad de cada punto.
- Calcular la densidad de la alcanzabilidad local
- Calcular el LOF

$$\text{Average RD}_A = \frac{1}{k}\sum_k \max[(k^{th}\text{ distance of A's neighbor, distance(A,}k^{th}\text{ neighbor)}]$$

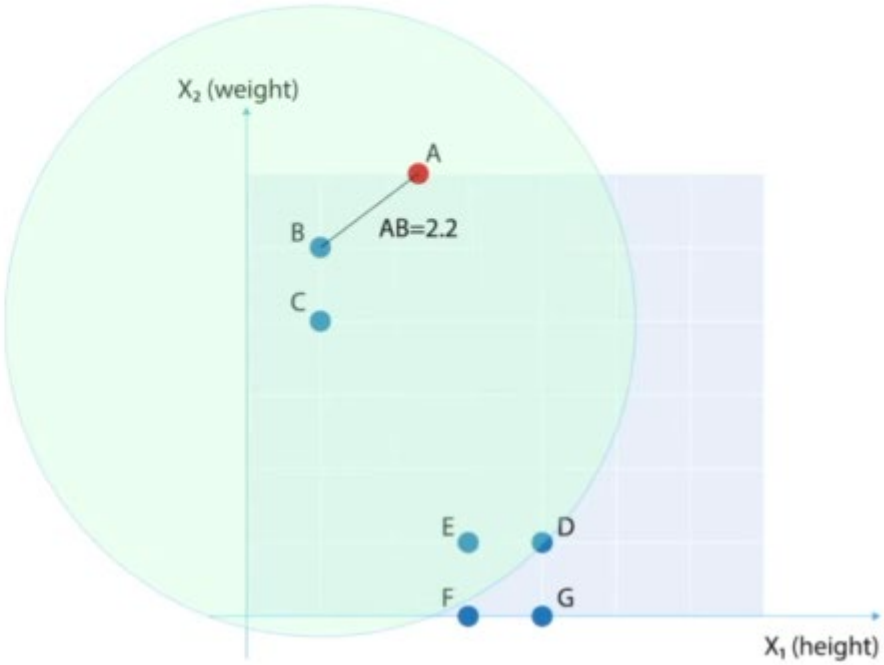
$$\Downarrow \; k=3$$

$$\text{Average RD}_A = \frac{1}{3}\sum_3 \max[(3^{rd}\text{ distance of A's neighbor, distance(A,}3^{rd}\text{ neighbor)}]$$

Scenario 1
A and B far from each other



Scenario 2
A and B close to each other



$$\text{Average RD}_A = \frac{1}{3}[\underbrace{\max(\overset{5}{3^{rd}\text{ dist B}}, \overset{6.08}{\text{dist(AB)}})}_{\text{Maximum of 5 and 6.08 = 6.08}} + \underbrace{\max(\overset{4.2}{3^{rd}\text{ dist C}}, \overset{6.32}{\text{dist(AC)}})}_{6.32} + \underbrace{\max(\overset{1.4}{3^{rd}\text{ dist D}}, \overset{5.8}{\text{dist(AD)}})}_{5.8}]$$

$$\text{Average RD}_A = \frac{1}{3}[6.08 + 6.32 + 5.8] = 6.06$$

Distances		
AB	6.08	2nd
AC	6.32	3rd
AD	5.8	1st
AE	6.4	
AF	7.2	
BA	6.08	
BC	1	1st
BD	5	3rd
BE	4.4	2nd
BF	5.3	
CA	6.32	
CB	1	1st
CD	4.2	3rd
CE	3.6	2nd
CF	4.4	
DA	5.8	
DB	5	
DC	4.2	
DE	1	1st
DF	1.4	3rd
DG	1	2nd

$$LRD_A = \frac{1}{RD_A}$$

1 $\text{Average}(RD_A) = \frac{1}{3}[\max(\overset{5}{3^{rd}\text{ dist B}}, \overset{6.08}{\text{dist(AB)}}) + \max(\overset{4.2}{3^{rd}\text{ dist C}}, \overset{6.32}{\text{dist(AC)}}) + \max(\overset{1.4}{3^{rd}\text{ dist D}}, \overset{5.8}{\text{dist(AD)}})] \longrightarrow 6.06$

$LRD_A = 0.164$

2 $\text{Average}(RD_B) = \frac{1}{3}[\max(\overset{6.32}{3^{rd}\text{ dist A}}, \overset{6}{\text{dist(AB)}}) + \max(\overset{4.2}{3^{rd}\text{ dist C}}, \overset{1}{\text{dist(BC)}}) + \max(\overset{1.4}{3^{rd}\text{ dist D}}, \overset{5}{\text{dist(BD)}})] \longrightarrow 5.17$

$LRD_B = 0.193$

3 $\text{Average}(RD_C) = \frac{1}{3}[\max(\overset{6.32}{3^{rd}\text{ dist A}}, \overset{6.32}{\text{dist(CA)}}) + \max(\overset{5}{3^{rd}\text{ dist C}}, \overset{1}{\text{dist(CB)}}) + \max(\overset{1}{3^{rd}\text{ dist D}}, \overset{4.2}{\text{dist(CD)}})] \longrightarrow 5.17$

$LRD_C = 0.193$

4 $\text{Average}(RD_D) = \frac{1}{3}[\max(\overset{6.32}{3^{rd}\text{ dist A}}, \overset{5.8}{\text{dist(DA)}}) + \max(\overset{5}{3^{rd}\text{ dist C}}, \overset{5}{\text{dist(DB)}}) + \max(\overset{4.2}{3^{rd}\text{ dist D}}, \overset{4.2}{\text{dist(DC)}})] \longrightarrow 5.17$

$LRD_D = 0.193$

$$LOF_A = \frac{\frac{1}{3}(LRD_B + LRD_C + LRD_D)}{LRD_A} = \frac{0.193}{0.164} = 1.17$$

Credit Card Fraud Detection Dataset | Kaggle

Resumen

- El bosque de aislamiento funciona muy bien.
- Los modelos de densidad son estupendos si son correctos.
- La estimación del ancho de banda puede ser complicada en el entorno no supervisado.
- La validación de los resultados suele requerir una inspección manual